

EXPERIMENT NO. 2

Title

Implementation of Method Overloading and Method Overriding using Inheritance in Java

Part A

Aim

To implement method overloading in Java and demonstrate compile-time polymorphism.

Software Requirements

Operating System: Windows / Linux / macOS

Programming Language: Java (JDK 8 or above)

IDE / Editor: IntelliJ IDEA / Eclipse / NetBeans / VS Code

Hardware Requirements

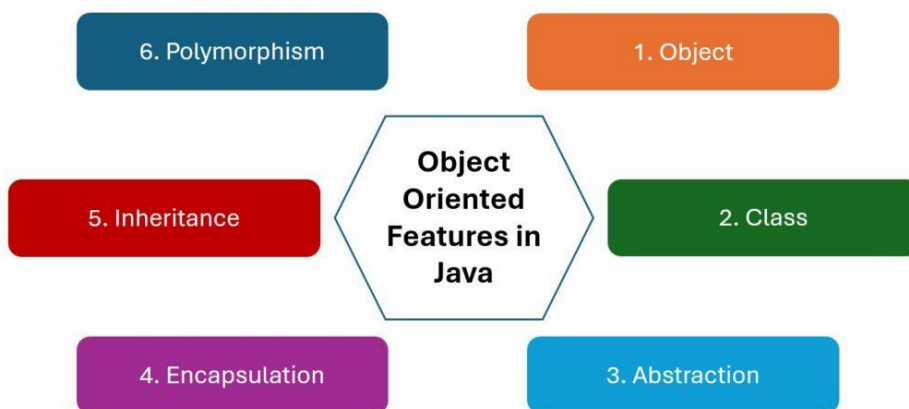
Processor: Intel i3 or higher

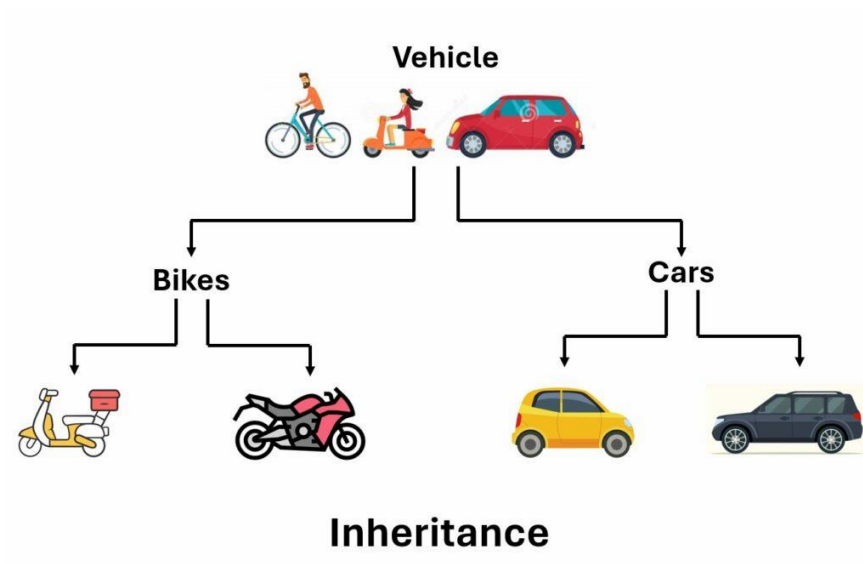
RAM: Minimum 4 GB

Hard Disk: 20 GB (minimum)

Introduction

Method overloading is a feature of Java that allows a class to have more than one method with the same name but different parameter lists.

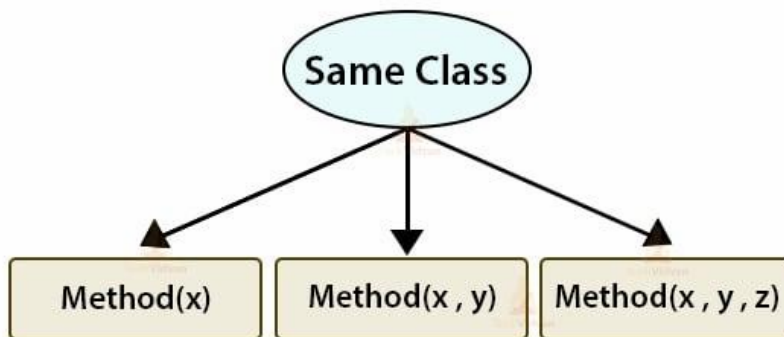




Theory

Method Overloading allows methods with the same name but different parameters. It is resolved at compile time and is known as compile-time polymorphism.

Method Overloading in Java



Algorithm

1. Start the program
2. Create a class Calculator
3. Define overloaded add() methods
4. Create object and call methods
5. Display result
6. Stop

Program Code

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
    double add(double a, double b) {  
        return a + b;  
    }  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class MethodOverloadingDemo {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(10, 20));  
        System.out.println(calc.add(10.5, 20.5));  
        System.out.println(calc.add(5, 10, 15));  
    }  
}
```

Output

30

31.0

30

(Attach screenshot of output window)

Learning Outcomes

Students will be able to understand and implement method overloading and compile-time polymorphism.

Conclusion

Thus, method overloading was successfully implemented in Java.

Part B

Aim

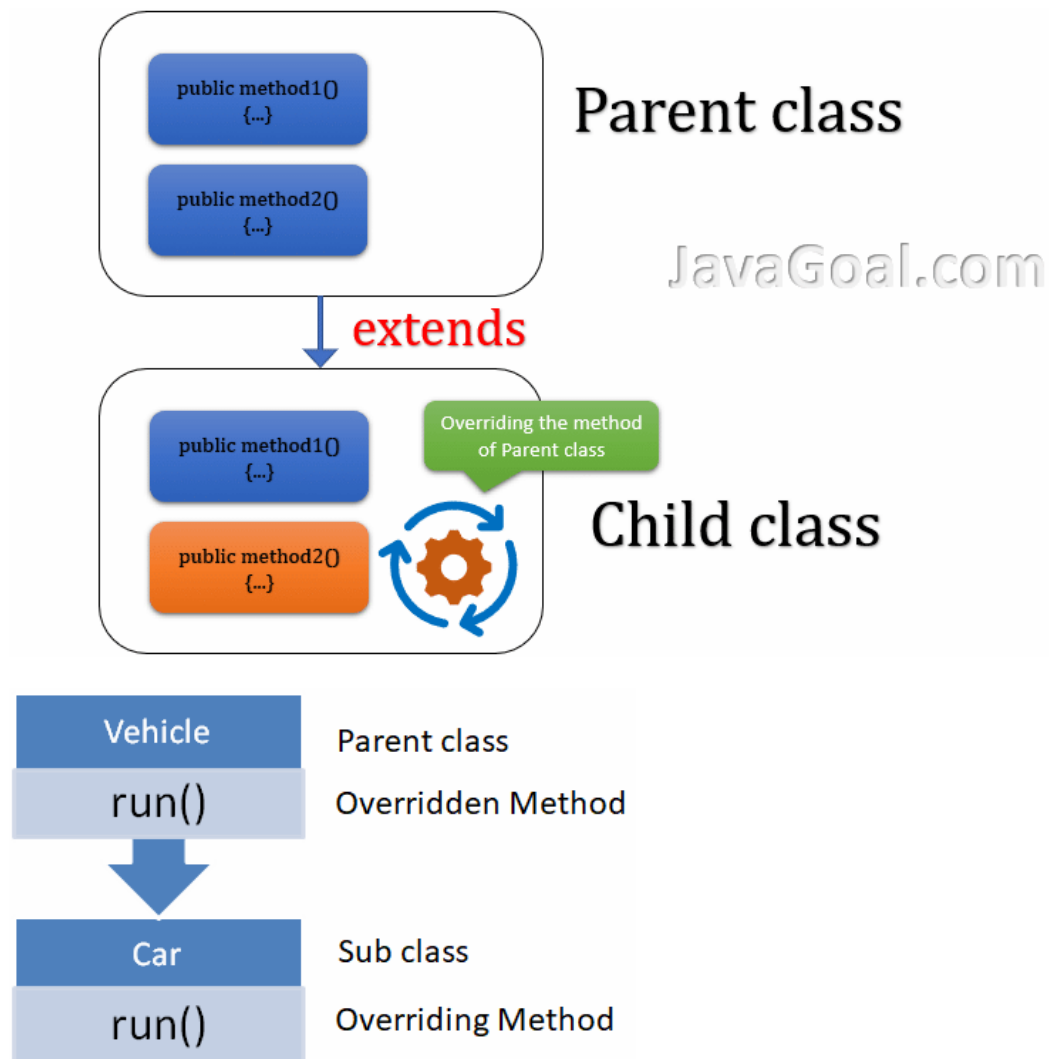
To implement method overriding using inheritance and demonstrate run-time polymorphism.

Introduction

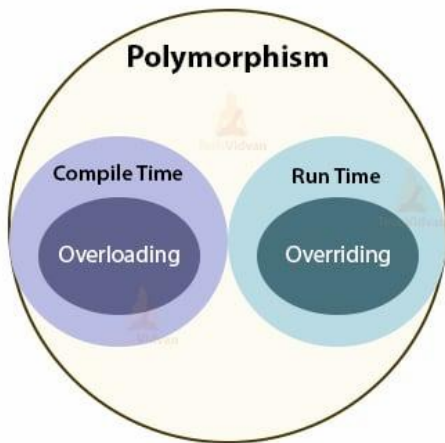
Method overriding allows a subclass to provide a specific implementation of a method already defined in the parent class.

Theory

Method overriding uses the same method signature and is resolved at runtime, therefore known as run-time polymorphism.



Comparison of Method Overloading & Overriding in Java



Algorithm

1. Start
2. Create parent class Vehicle
3. Create child class Car
4. Override speed() method
5. Display output
6. Stop

Program Code

```
class Vehicle
{
    void speed()
    {
        System.out.println("Vehicle speed is moderate");
    }
}
class Car extends Vehicle
{
    @Override
    void speed()
    {
        System.out.println("Car speed is fast");
    }
}
public class MethodOverridingDemo
{
    public static void main(String[] args)
    {
```

```
Vehicle v = new Car();  
v.speed();  
}  
}
```

Output

Car speed is fast

(Attach screenshot of output window)

Learning Outcomes

Students will understand inheritance and runtime polymorphism.

Conclusion

Hence, method overriding using inheritance was successfully implemented.